Course: Artificial Intelligence and Machine Learning **Code: 20CS51I**

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 1

11.1 UNDERSTANDING NATURAL LANGUAGE PROCESSING

Language is the primary means of communication used by humans. It is the tool we use to express the greater part of our ideas and emotions. It shapes thought, has a structure, and carries meaning. Learning new concepts and expressing ideas through them is so natural that we hardly realize how we process natural language.

"A natural language or ordinary language is any language that has evolved naturally in humans through use and repetition without conscious planning or premeditation." In contrast to artificial languages like Python, C, Java, etc. natural languages like English, Kannada, Hindi, French, etc. have evolved over time and use and it is difficult to express them in strict formal rules.

"Natural Language Processing (NLP) is a branch of Artificial Intelligence that helps computers to understand, interpret and manipulate human languages to analyze and derive its meaning."

NLP incorporates **machine learning models, statistics, and deep learning models** into **computational linguistics** i.e., rule-based modeling of human language to allow computers to understand text, spoken words and understands human language, intent, and sentiment. It helps developers to organize and structure knowledge to perform tasks like translation, summarization, named entity recognition, relationship extraction, speech recognition, topic segmentation, etc.

11.1.1 Why we need NLP?

Natural language is full of **ambiguities.** Ambiguity can be referred to as the ability of having more than one meaning or being understood in more than one way.

There are different types of ambiguities present in natural language:

1. Lexical Ambiguity: It is defined as the ambiguity associated with the meaning of a single word. A single word can have different meanings. Also, a single word can be a noun, adjective, or verb. For example: The word "bank" can have different meanings. It can be a financial bank or a riverbank. Similarly, the word "clean" can be a noun, adverb, adjective, or verb.

2. Syntactic Ambiguity: It is defined as the ambiguity associated with the way the words are parsed. For example: The sentence "Visiting relatives can be boring." This sentence can have two different meanings. One is that visiting a relative's house can be boring. The second is that visiting relatives at your place can be boring.

3. Semantic Ambiguity: It is defined as ambiguity when the meaning of the words themselves can be ambiguous. For example: The sentence "Mary knows a little French." In this sentence the word "little French" is ambiguous. As we don't know whether it is about the language French or a person.

Let us now try to understand why NLP is considered hard using a few examples.

1. "There was not a single man at the party"
   - Does it mean that there were no men at the party? or
   - Does it mean that there was no one at the party?
   - Here does man refer to the gender "man" or "mankind"?
2. "The chicken is ready to eat"
   - Does this mean that the bird (chicken) is ready to feed on some grains? or
   - Does it mean that the meat is cooked well and is ready to be eaten by a human?
3. "Google is a great company." and "Google this word and find its meaning."
   - Google is being used as a noun in the first statement and as a verb in the second.
4. The man saw a girl with a telescope.
   - Did the man use a telescope to see the girl? or
   - Did the man see a girl who was holding a telescope?

This is a primary reason why NLP is considered hard. Another reason why NLP is hard is because it deals with the extraction of knowledge from unstructured data.

11.2 NLP APPROACHES

11.2.1 Rule Based NLP

Rule-based approaches are the oldest approaches to NLP. The rule-based or grammar-based approach implies that a human is involved in the process of stepwise system development and improvement. A rule-based NLP system simply follows these rules to categorize the language it's analyzing. If the rule doesn't exist, the system will be unable to 'understand' the human language and thus will fail to categorize it. Regular expressions and context free grammars are some examples of rule-based approaches to NLP.

Advantages of Rule based approach:

- A rule-based system is good at capturing a specific language phenomenon: it will decode the linguistic relationships between words to interpret the sentence.
- It tends to focus on pattern-matching or parsing.

- Rule-based systems are low precision, high recall, meaning they can have high performance in specific use cases, but often suffer performance degradation when generalized.

Disadvantage of Rule-based approach:

- It requires skilled experts: it takes a linguist or a knowledge engineer to manually encode each rule in NLP.
- Rules need to be manually crafted and enhanced all the time.
- Moreover, the system can become so complex, that some rules can start contradicting each other.
- Accuracy of the NLP system is dependent on the rules provided
- They cannot easily scale to accommodate a seemingly endless stream of exceptions or the increasing volumes of text and voice data.

11.2.2 Statistical NLP

Statistical NLP aims to perform statistical inference for the field of NLP. It combines computer algorithms with machine learning and deep learning models to automatically extract, classify, and label elements of text and voice data and then assign a statistical likelihood to each possible meaning of those elements. Today, deep learning models and learning techniques based on convolutional neural networks (CNNs) and recurrent neural networks (RNNs) enable NLP systems that 'learn' as they work and extract ever more accurate meaning from huge volumes of raw, unstructured, and unlabeled text and voice data sets.

Advantages of Statistical NLP:

- The main advantage of statistical NLP is machine learning algorithm's "learnability", which is why no manual rule/grammar coding is needed, requiring high skills.
- The corpus can be annotated using the low-skilled workforce.
- The data fed to such system will be huge and there are a lot of data points (e.g. keywords etc.), which makes it easy for the machine to learn statistical clues of the words for a given task.
- Machine learning approaches can significantly speed up the development of a capability of certain NLP systems, when good training data sets are available

Disadvantages of Statistical NLP:

- Lack of training data
- Poorly labelled, insufficient data
- New "preparation" of data is required each time as, once created and labelled, the corpus often can't be reused on new data schemas.

11.3 NLP USE CASES

Natural language processing is the driving force behind machine intelligence in many modern real-world applications. Here are a few examples:

- **Spam detection:** NLP's text classification capabilities can be used to scan emails for language that often indicates spam or phishing. These indicators can include overuse of financial terms, characteristic bad grammar, threatening language, inappropriate urgency, misspelled company names, and more.
- **Machine translation:** Machine translation involves translating words/sentences in one language to another. Effective translation has to capture accurately the meaning and tone of the input language and translate it to text with the same meaning and desired impact in the output language. Example: Google translate
- **Speech Recognition:** This is the process of mapping acoustic speech signals to a set of words. Difficulty arises due to wide variations in the pronunciations of words, homonym (e.g., dear and deer) and acoustic ambiguities (e.g., in the rest and interest)
- **Speech Synthesis:** It refers to automatic production of speech (uttering sentences in natural language). Such systems can read out your mail or messages for you.
- **Virtual agents and chatbots:** Virtual agents such as Apple's Siri and Amazon's Alexa use speech recognition to recognize patterns in voice commands and natural language generation to respond with appropriate action or helpful comments. Chatbots perform the same magic in response to typed text queries.
- **Social media sentiment analysis:** Sentiment analysis can analyze language used in social media posts, responses, reviews, and more to extract attitudes and emotions in response to products, promotions, and events. companies can use this information in product designs, advertising campaigns, and more.
- **Text summarization:** Text summarization uses NLP techniques to digest huge volumes of digital text and create summaries and synopses for indexes, research databases, or busy readers who don't have time to read full text. The best text summarization applications use semantic reasoning and natural language generation (NLG) to add useful context and conclusions to summaries.

11.4 NLP TOOLS & LIBRARIES

Some commonly used NLP tools and libraries are as follows:

1. NLTK - entry-level open-source NLP Tool

Natural Language Toolkit (AKA NLTK) is an open-source software powered with Python NLP. NLTK provides users with a basic set of tools for text-related operations. It is a good starting point for beginners in Natural Language Processing.

Natural Language Toolkit features include:

- Text classification
- Part-of-speech tagging
- Entity extraction
- Tokenization
- Parsing
- Stemming
- Semantic reasoning

Natural Language Toolkit is useful for simple text analysis. But, if you need to work on a massive amount of data, it requires significant resources.

2. Stanford Core NLP - Data Analysis, Sentiment Analysis, Conversational UI

Stanford NLP library is a multi-purpose tool for text analysis. Like NLTK, Stanford CoreNLP provides many different natural language processing software. But if you need more, you can use custom modules. The main advantage of Stanford NLP tools is scalability. Unlike NLTK, Stanford Core NLP is a perfect choice for processing large amounts of data and performing complex operations.

3. Apache OpenNLP - Data Analysis and Sentiment Analysis

Apache OpenNLP is an open-source library for those who prefer practicality and accessibility. Like Stanford CoreNLP, it uses Java NLP libraries with Python decorators. While NLTK and Stanford CoreNLP are state-of-the-art libraries with tons of additions, OpenNLP is a simple yet useful tool. Besides, you can configure OpenNLP in the way you need and get rid of unnecessary features.

Apache OpenLP is the right choice for:

- Named Entity Recognition
- Sentence Detection
- POS tagging
- Tokenization

4. SpaCy - Data Extraction, Data Analysis, Sentiment Analysis, Text Summarization

SpaCy is the next step of the NLTK evolution. NLTK is clumsy and slow when it comes to more complex business applications. At the same time, SpaCy provides users with a smoother, faster, and efficient experience.

- SpaCy, an open-source NLP library, is a perfect match for comparing customer profiles, product profiles, or text documents.
- SpaCy is good at syntactic analysis, which is handy for aspect-based sentiment analysis and conversational user interface optimization.

- SpaCy is also an excellent choice for named-entity recognition. You can use SpaCy for business insights and market research.
- Another SpaCy advantage is word vector usage. Unlike OpenNLP and CoreNLP, SpaCy works with word2vec and doc2vec.

5. GenSim - Document Analysis, Semantic Search, Data Exploration

GenSim is the perfect tool to extract particular information to discover business insights. It is an open-source NLP library designed for document exploration and topic modeling. It would help you to navigate the various databases and documents.

- The key GenSim feature is word vectors. It sees the content of the documents as sequences of vectors and clusters, and then, classifies them.
- GenSim is also resource-saving when it comes to dealing with a large amount of data.

The main GenSim use cases are:

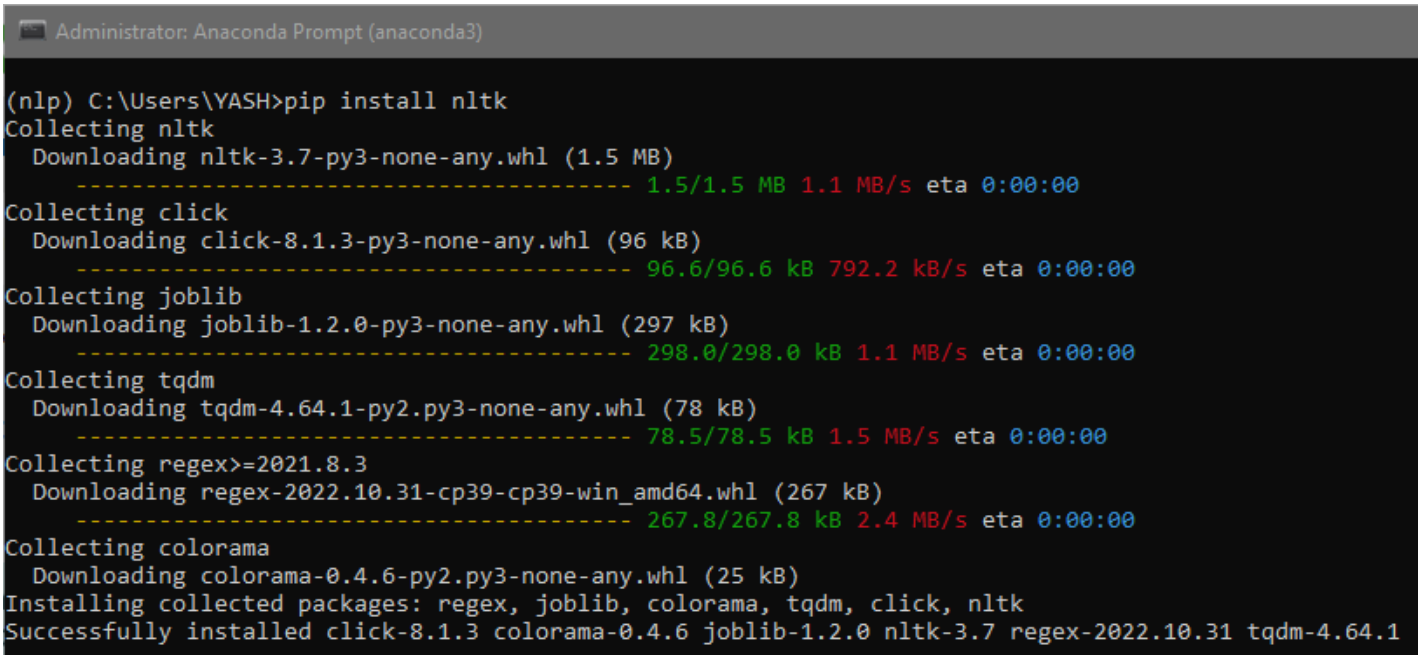
- Data analysis
- Semantic search applications
- Text generation applications (chatbot, service customization, text summarization, etc.)

11.5 ENVIRONMENT SETUP

NLTK can be installed by using the pip package installer. Recently NLTK has dropped support for Python 2 so Python 3.5 and above is required to install NLTK.

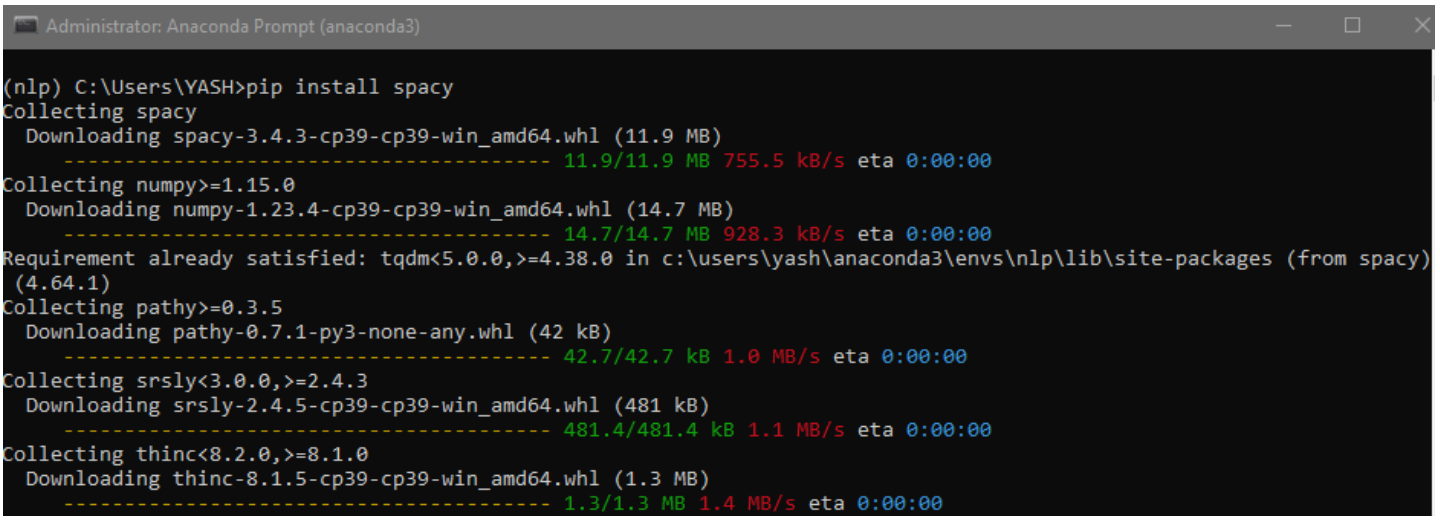
Pip command to install NLTK is as follows:

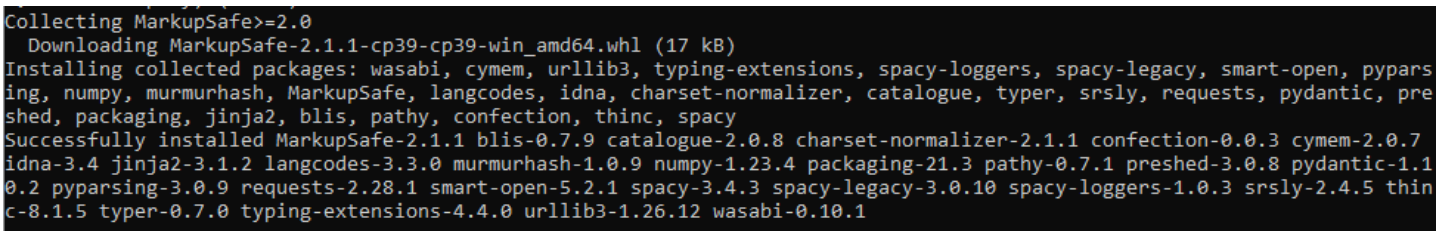
```
pip install nltk
```



We can install SpaCy in a similar way using

pip install spacy





References:

1. Tanveer Siddiqui, U.S. Tiwary, “Natural Language Processing and Information Retrieval”, Oxford University Press, 2008.
2. https://medium.com/friendly-data/machine-learning-vs-rule-based-systems-in-nlp-5476de53c3b8
3. https://www.sentisum.com/success-article/machine-learning-nlp
4. https://www.kdnuggets.com/2018/10/main-approaches-natural-language-processing-tasks.html
5. https://www.ibm.com/cloud/learn/natural-language-processing
6. https://theappsolutions.com/blog/development/nlp-tools/
7. Infosys Springboard – Natural Language Processing for developers

Course: Artificial Intelligence and Machine Learning **Code: 20CS51I**

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 2

11.6 TEXT PROCESSING TASKS

Text data derived from natural language is unstructured and noisy. Text preprocessing involves transforming text into a clean and consistent format that can then be fed into a model for further analysis and learning.

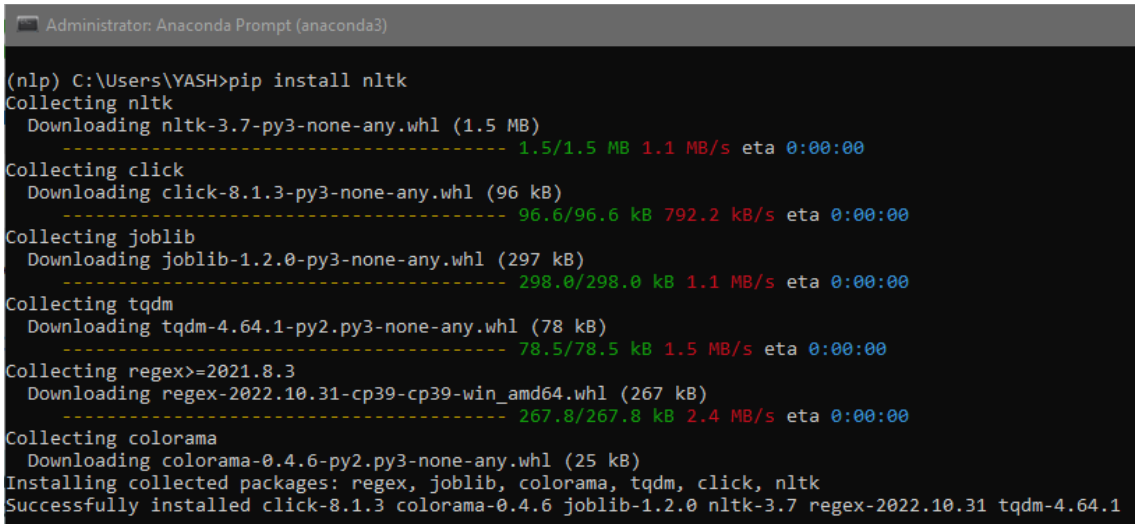
- Text preprocessing is an important step for natural language processing (NLP) tasks.
- It transforms text into a more digestible form so that machine learning algorithms can perform better, and depending on how well the data has been preprocessed; the results are seen.
- Text preprocessing improves the performance of an NLP system.
- For tasks such as sentiment analysis, document categorization, document retrieval based upon user queries, and more, adding a text preprocessing layer provides more accuracy.

Some of the preprocessing steps are:

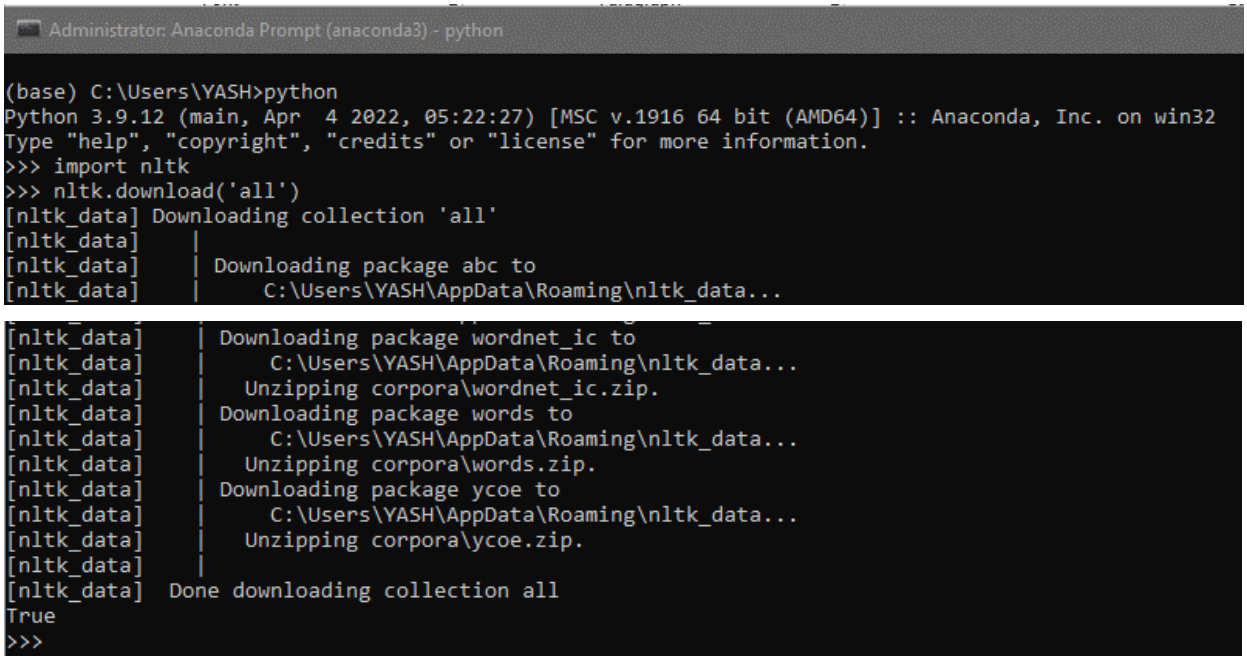
- Tokenization
- Removing Stop words
- Spelling correction
- Stemming
- Lemmatization

We will be using NLTK to perform all NLP tasks. If NLTK is not yet installed in your system, go to Anaconda Command prompt and perform the following steps:

- **pip install nltk**



- Then, enter the python shell in your terminal by simply typing **python**
- Type **import nltk**
- Type **nltk.download('all')**



The above installation will take quite some time due to the massive number of tokenizers, chunkers, other algorithms, and all of the corpora to be downloaded.

11.6.1 Tokenization

Token in a text document refers to each "entity" that is a part of whatever was split up based on rules. For examples, each word is a token when a sentence is "tokenized" into words. Each sentence can also be a token, if you tokenized the sentences out of a paragraph. So basically, tokenizing involves **splitting sentences and words from the body of the text.**

NLTK Tokenizer Package

The nltk.tokenize package contains methods to tokenize the given text into sentences and words based on our requirement.

1. Sentence tokenization

The given document is divided or tokenized into sentences.

Syntax:

nltk.tokenize.sent_tokenize(text, language='english')

returns a sentence-tokenized copy of text, using NLTK's recommended sentence tokenizer. (currently PunktSentenceTokenizer for the specified language).

Parameters:

Text (str) – text to split into sentences

Language(str) – the model name in the Punkt corpus. By default it will be English.

Example:

The following example reads a text file and tokenizes it into sentences.

- Import the sent_tokenize method from nltk.tokenize package

```
from nltk.tokenize import sent_tokenize
```

- Read the text file using open method

```
file = open("nlp.txt", "r")
text = file.read()
print(text)
```

Output:

```
Natural language processing (NLP) refers to the branch of computer science and
more specifically, the branch of artificial intelligence or AI, concerned with
giving computers the ability to understand text and spoken words in much the sa
me way human beings can.

NLP combines computational linguistics with statistical, machine learning, and
deep learning models. Together, these technologies enable computers to process
human language in the form of text or voice data and to understand its full mea
ning, complete with the speaker or writer's intent and sentiment.
```

- Divide the text into sentences using the sent_tokenize method

```
sentences = sent_tokenize(text)
```

- Print the sentences to see how the method has divided the text into sentences.

```
print("Number of sentences:",len(sentences))
for i in range(len(sentences)):
    print("\nSentence ",i+1,":\n",sentences[i])
```

Output:

```
Number of sentences: 3

Sentence  1 :
 Natural language processing (NLP) refers to the branch of computer science and
more specifically, the branch of artificial intelligence or AI, concerned with
giving computers the ability to understand text and spoken words in much the sa
me way human beings can.

Sentence  2 :
 NLP combines computational linguistics with statistical, machine learning, and
deep learning models.

Sentence  3 :
 Together, these technologies enable computers to process human language in the
form of text or voice data and to understand its full meaning, complete with th
e speaker or writer's intent and sentiment.
```

2. Word tokenization

The given document is divided or tokenized into words.

Syntax:

nltk.tokenize.word_tokenize(text, language='english', preserve_line=False)

returns a tokenized copy of text, using NLTK's recommended word tokenizer (currently an improved TreebankWordTokenizer along with PunktSentenceTokenizer for the specified language).

Parameters

text (str) – text to split into words

language (str) – the model name in the Punkt corpus

preserve_line (bool) – A flag to decide whether to sentence tokenize the text or not.

Example:

For the text file used in previous example, let us find the word tokens.

```
from nltk.tokenize import word_tokenize
```

```
words = word_tokenize(text)
```

```
print("Total number of words: ",len(words))
print(words)
```

Output:

```
Total number of words:  97
['Natural', 'language', 'processing', '(', 'NLP', ')', 'refers', 'to', 'the', 'branch', 'of', 'comput
er', 'science', 'and', 'more', 'specifically', ',', 'the', 'branch', 'of', 'artificial', 'intelligenc
e', 'or', 'AI', ',', 'concerned', 'with', 'giving', 'computers', 'the', 'ability', 'to', 'understan
d', 'text', 'and', 'spoken', 'words', 'in', 'much', 'the', 'same', 'way', 'human', 'beings', 'can',
'.', 'NLP', 'combines', 'computational', 'linguistics', 'with', 'statistical', ',', 'machine', 'learn
ing', ',', 'and', 'deep', 'learning', 'models', '.', 'Together', ',', 'these', 'technologies', 'enabl
e', 'computers', 'to', 'process', 'human', 'language', 'in', 'the', 'form', 'of', 'text', 'or', 'voic
e', 'data', 'and', 'to', 'understand', 'its', 'full', 'meaning', ',', 'complete', 'with', 'the', 'spe
aker', 'or', 'writer', "'s", 'intent', 'and', 'sentiment', '.']
```

The tokens generated will contain special characters such as comma, dot, parentheses, apostrophe etc. We may have to remove these special characters, and stop words to generate the vocabulary of our text document.

3. Visualizing Frequency Distribution of words

The standard method for visualizing the word frequency distribution is to count how often each word occurs in a corpus and to sort the word frequency counts by decreasing magnitude. We need to create a Bag of Words (BoW), which is a statistical language model used to analyze text and documents based on word count.

Example:

To visualize the frequency distribution, let us consider opinions given by customers about a hotel.

- Read the "Opinion.txt" text file which contains opinions given by multiple customers

```
file = open("opinion.txt", "r")
text = file.read()
```

```
words = word_tokenize(text)
```

- **Creating the Frequency Distribution**

  Without the NLTK package, creating a frequency distribution plot (histogram) for a BoW is possible, but will take multiple lines of code to do so. Through the use of the FreqDist class, we are able to obtain the frequencies of every token in the BoW with one single line of code:

```
from nltk.probability import FreqDist

## Creating FreqDist for whole BoW, keeping the 20 most common tokens
all_fdist = FreqDist(words)
all_fdist['service']
```

Output:

```
[('the', 77),
 ('food', 52),
 ('and', 41),
 ('a', 29),
 ('was', 26),
 ('for', 19),
 ('to', 15),
 ('of', 14),
 ('we', 13),
 ('service', 13),
 ('very', 12),
 ('i', 12),
 ('at', 12),
 ('in', 10),
 ('is', 10),
 ('room', 10),
 ('but', 9),
 ('our', 8),
 ('hotel', 8),
 ('with', 8)]
```

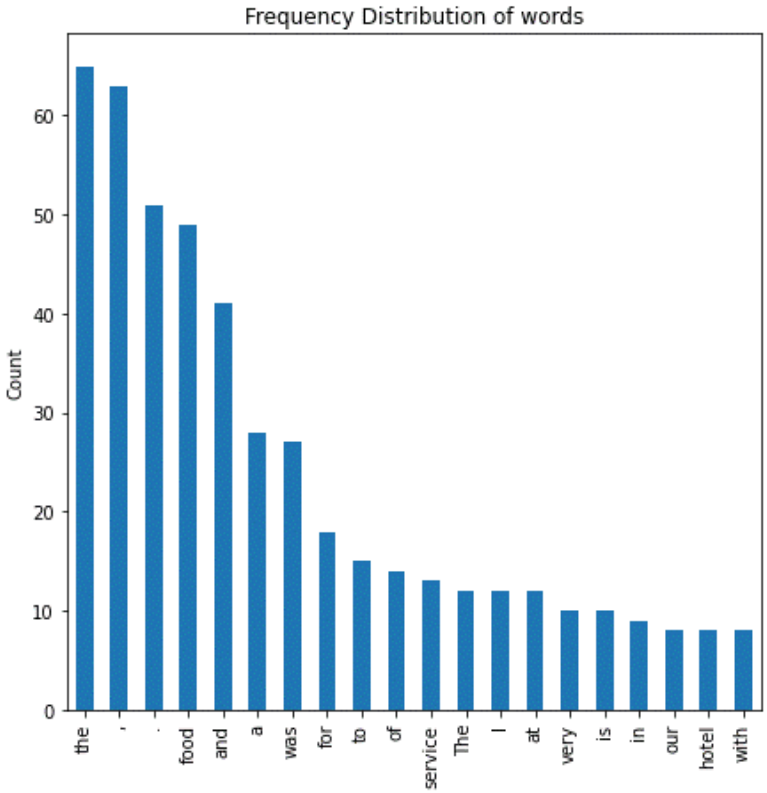
- **Visualizing Frequency Distribution using matplotlib**

```
import matplotlib.pyplot as plt
import pandas as pd

## Conversion to Pandas series via Python Dictionary for easier plotting
all_fdist = pd.Series(dict(all_fdist))

## Setting figure, ax into variables
fig, ax = plt.subplots(figsize=(10,10))

all_fdist.plot(kind='bar')
plt.title("Frequency Distribution of words")
plt.ylabel("Count")
```



As we can see, the frequency distribution contains frequencies of tokens like 'the', 'to', 'of' which are considered as stop words. It also contains special characters and tokens with different case are treated differently. To overcome these issues, we have to perform text cleanup as follows:

- **Converting the text to lower case letters.**

```
text =text.lower()
```

- **Removing special character from data**

```
import re
text = re.sub('[^A-Za-z0-9]+', ' ', text)
```

- **Removing words with numbers from data**

```
text = re.sub("\S*\d\S*", "", text).strip()
```

- **Removing Stopwords**

Stop words are a set of commonly used words in any language. For example, in English, "the", "is" and "and", would easily qualify as stop words. In NLP and text mining applications, stop words are used to eliminate unimportant words, allowing applications to focus on the important words instead.

NLTK has an inbuilt list of stopwords. We can also create our own stopword list and use it based on our requirements. The following code removes the stopwords from the text. When we visualize the frequency distribution after text cleaning, it will be as follows:

```
import nltk
words = word_tokenize(text)
stopwords = nltk.corpus.stopwords.words('english')
words_sw_removed = []

## Check if token in stop word list before adding to new list
for word in words:
    if word in stopwords:
        pass
    else:
        words_sw_removed.append(word)
```

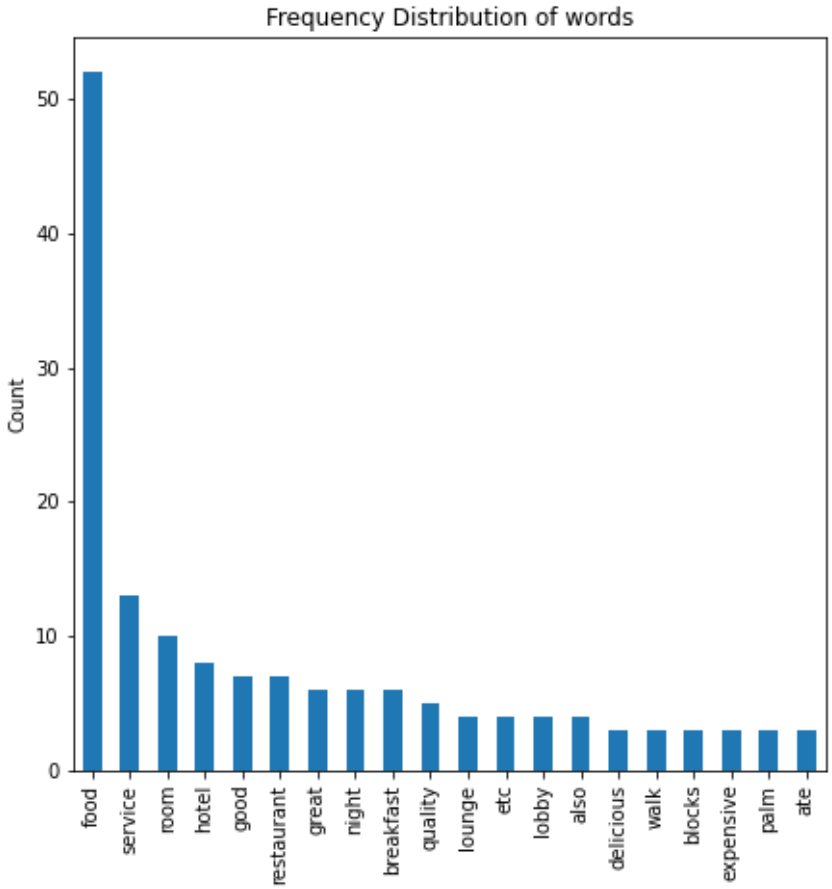
When we draw the Histogram after the text cleaning, it would look like the below figure.

```
all_fdist = FreqDist(words_sw_removed).most_common(20)

## Conversion to Pandas series via Python Dictionary for easier plotting
all_fdist = pd.Series(dict(all_fdist))

## Setting figure, ax into variables
fig, ax = plt.subplots(figsize=(7,7))

all_fdist.plot(kind='bar')
plt.title("Frequency Distribution of words")
plt.ylabel("Count")
```



4. Visualize with word cloud

A word cloud is a collection, or cluster, of words depicted in different sizes. The bigger and bolder the word appears, the more often it's mentioned within a given text and the more important it is.

To draw a word cloud, install it through pip command as follows:

pip install wordcloud

Example:

```
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

stopwords = set(STOPWORDS)

wordcloud = WordCloud(width = 800, height = 800,
                      background_color ='white',
                      stopwords = stopwords,
                      min_font_size = 10).generate(text)

# plot the WordCloud image
plt.figure(figsize = (8, 8), facecolor = None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad = 0)

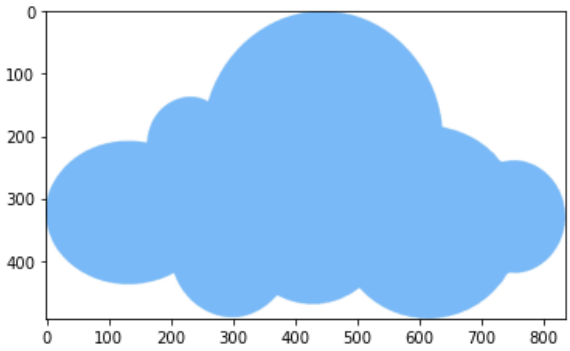
plt.show()
```

Output:



We can set a custom image as a mask to display the word cloud as follows:

```
#reading an image to set the custom outline of Wordcloud
from skimage.io import imread
cloud = imread('cloud.png')
plt.imshow(cloud)
```




```
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

stopwords = set(STOPWORDS)

wordcloud = WordCloud(width = 800, height = 800,
                      background_color ='white',
                      stopwords = stopwords,
                      min_font_size = 10,mask=cloud).generate(text)

# plot the WordCloud image
plt.figure(figsize = (8, 8), facecolor = None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad = 0)

plt.show()
```



References:

1. https://www.nltk.org/api/nltk.tokenize.html
2. https://basilkjose.medium.com/data-preprocessing-natural-language-competition-processing
3. https://towardsdatascience.com/text-preprocessing-in-natural-language-processing-using-python-6113ff5decd8
4. Infosys Springboard – Natural Language Processing for developers

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 3

11.7 SPELL CORRECTION

Spelling correction is the process of correcting word's spelling. For example "lisr" instead of "list". Spelling correction is important for many NLP applications like web search engines, text summarization, sentiment analysis etc. Most approaches use parallel data of noisy and correct word mappings from different sources as training data for automatic spelling correction.

Here we are going to **use Levenshtein distance or Edit Distance** method for spelling correction. This method takes a list of misspelled words and gives the suggestion of the correct word for each incorrect word. It tries to find a word in the list of correct spellings that has the shortest distance and the same initial letter as the misspelled word. It then returns the word which matches the given criteria.

11.7.1 Edit Distance

Edit Distance or Levenshtein distance between two words is the minimum number of single-character edits (insertions, deletions or substitutions) required to change one word into the other.

Edit Distance measures dissimilarity between two strings by finding the minimum number of operations needed to transform one string into the other. The transformations that can be performed are:

- Inserting a new character:
  bat -> bats (insertion of 's')
- Deleting an existing character.
  care -> car (deletion of 'e')
- Substituting an existing character.
  bin -> bit (substitution of n with t)
- Transposition of two existing consecutive characters.
  sing -> sign (transposition of ng to gn)

Implementation using NLTK

Step 1: First of all, we install and import the nltk suite.

```
import nltk
# importing edit distance
from nltk.metrics.distance import edit_distance
```

Step 2: Now, we download the 'words' resource (which contains correct spellings of words) from the nltk downloader and import it through nltk.corpus and assign it to correct_words.

```
# Downloading and importing package 'words' if not already installed
nltk.download('words')
from nltk.corpus import words
correct_words = words.words()
```

Step 3: We define the list of incorrect_words for which we need the correct spellings. Then we run a loop for each word in the incorrect words list in which we calculate the Edit distance of the incorrect word with each correct spelling word having the same initial letter. We then sort them in ascending order so the shortest distance is on top and extract the word corresponding to it and print it.

```
# list of incorrect spellings
# that need to be corrected
incorrect_words=['happpy', 'amazaing', 'intelliengt']

# loop for finding correct spellings
# based on edit distance and
# printing the correct words
for word in incorrect_words:
    temp = [(edit_distance(word, w),w) for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

```
happy
amazing
intelligent
```

11.8 NORMALIZATION

Base form of a word is defined as **Morpheme**. A token is basically made up of two components one is morpheme and the other is inflectional form like prefix or suffix. For example, consider the word Antinationalist (Anti + national+ ist) which is made up of Anti and ist as inflectional forms and national as the morpheme.

Normalization is the process of converting a token into its base form. In the normalization process, the inflectional form of a word is removed so that the base form can be obtained. In the above example, the normal form of antinationalist is national.

Why do we need text normalization?

- Normalization is helpful in reducing the number of unique tokens present in the text, removing the variations in a text and also cleaning the text by removing redundant information.
- When we normalize text, we attempt to reduce its randomness, bringing it closer to a predefined "standard".

- This helps us to reduce the amount of different information that the computer has to deal with, and therefore improves efficiency.

Two popular methods used for normalization are

- Stemming
- Lemmatization

11.8.1 Stemming

Stemming is the process of reducing the words to their root form. It is a rule-based process for removing inflationary forms from a given token. The output of this process is called the **stem**. For example, “retrieval”, “retrieved”, “retrieves” reduce to the stem “retrieve”. Another example of stemming can be "likes", "liked", "likely", "liking" are reduced to root word "like".

The objective of stemming is to reduce related words to the same stem even if the stem is not a dictionary word. Stemming is not a good process for normalization. since sometimes it can produce non-meaningful words which are not present in the dictionary.

Implementation using Porter Stemmer

The most common algorithm for stemming English is **Porter’s algorithm**. There are other stemmers also available such as Snowball Stemmer.

Step 1: First of all, we read the text from a file and perform text cleaning such as converting to lower case, removing punctuations and numeric data from the text.

```
from nltk.tokenize import word_tokenize

file = open("nlp.txt", "r")
text = file.read()
text =text.lower()
import re
text = re.sub('[^A-Za-z0-9]+', ' ', text)
text = re.sub("\S*\d\S*", "", text).strip()
print(text)
```

```
natural language processing nlp refers to the branch of computer science and mo
re specifically the branch of artificial intelligence or ai concerned with givi
ng computers the ability to understand text and spoken words in much the same w
ay human beings can nlp combines computational linguistics with statistical mac
hine learning and deep learning models together these technologies enable compu
ters to process human language in the form of text or voice data and to underst
and its full meaning complete with the speaker or writer s intent and sentiment
```

Step 2: The text is then split into tokens or words and each word is fed to the Porter Stemmer to get the stem word.

```
words = word_tokenize(text,preserve_line=True)
print(words)
```

```
['natural', 'language', 'processing', 'nlp', 'refers', 'to', 'the', 'branch',
'of', 'computer', 'science', 'and', 'more', 'specifically', 'the', 'branch', 'o
f', 'artificial', 'intelligence', 'or', 'ai', 'concerned', 'with', 'giving', 'c
omputers', 'the', 'ability', 'to', 'understand', 'text', 'and', 'spoken', 'word
s', 'in', 'much', 'the', 'same', 'way', 'human', 'beings', 'can', 'nlp', 'combi
nes', 'computational', 'linguistics', 'with', 'statistical', 'machine', 'learni
ng', 'and', 'deep', 'learning', 'models', 'together', 'these', 'technologies',
'enable', 'computers', 'to', 'process', 'human', 'language', 'in', 'the', 'for
m', 'of', 'text', 'or', 'voice', 'data', 'and', 'to', 'understand', 'its', 'ful
l', 'meaning', 'complete', 'with', 'the', 'speaker', 'or', 'writer', 's', 'inte
nt', 'and', 'sentiment']
```

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
ps_stem_sent = [ps.stem(words_sent) for words_sent in words]
print(ps_stem_sent)
```

```
['natur', 'languag', 'process', 'nlp', 'refer', 'to', 'the', 'branch', 'of', 'c
omput', 'scienc', 'and', 'more', 'specif', 'the', 'branch', 'of', 'artifici',
'intellig', 'or', 'ai', 'concern', 'with', 'give', 'comput', 'the', 'abil', 't
o', 'understand', 'text', 'and', 'spoken', 'word', 'in', 'much', 'the', 'same',
'way', 'human', 'be', 'can', 'nlp', 'combin', 'comput', 'linguist', 'with', 'st
atist', 'machin', 'learn', 'and', 'deep', 'learn', 'model', 'togeth', 'these',
'technolog', 'enabl', 'comput', 'to', 'process', 'human', 'languag', 'in', 'th
e', 'form', 'of', 'text', 'or', 'voic', 'data', 'and', 'to', 'understand', 'i
t', 'full', 'mean', 'complet', 'with', 'the', 'speaker', 'or', 'writer', 's',
'intent', 'and', 'sentiment']
```

As we can see, some words are converted to a correct root word, such as ‘refers’ is reduced to ‘refer’, ‘processing’ is reduced to ‘process’. But most of the words are stemmed to a word which is not present in the dictionary and does not carry any meaning. **Stemming a word or sentence may result in words that are not actual words.**

11.8.2 Lemmatization

Lemmatization is a systematic process of removing the inflectional form of a token and transform it into a base word. It makes use of word structure, vocabulary, part of speech tags, and grammar relations.

Unlike stemming, lemmatization reduces words to their base word, reducing the inflected words properly and ensuring that the root word belongs to the language. It’s usually more sophisticated than stemming,

since stemmers works on an individual word without knowledge of the context. In lemmatization, a root word is called **lemma**. For example, “am”, “are”, “is” will be converted to “be”. Similarly, ‘running’, ‘runs’, ‘ran’ will be replaced by ‘run’.

Just like for stemming, there are different lemmatizers. Here we are going to use WordNet lemmatizer.

```
from nltk.stem.wordnet import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lem_sent = [lemmatizer.lemmatize(words_sent) for words_sent in words]
print(lem_sent)
```

```
['natural', 'language', 'processing', 'nlp', 'refers', 'to', 'the', 'branch',
'of', 'computer', 'science', 'and', 'more', 'specifically', 'the', 'branch', 'o
f', 'artificial', 'intelligence', 'or', 'ai', 'concerned', 'with', 'giving', 'c
omputer', 'the', 'ability', 'to', 'understand', 'text', 'and', 'spoken', 'wor
d', 'in', 'much', 'the', 'same', 'way', 'human', 'being', 'can', 'nlp', 'combin
e', 'computational', 'linguistics', 'with', 'statistical', 'machine', 'learnin
g', 'and', 'deep', 'learning', 'model', 'together', 'these', 'technology', 'ena
ble', 'computer', 'to', 'process', 'human', 'language', 'in', 'the', 'form', 'o
f', 'text', 'or', 'voice', 'data', 'and', 'to', 'understand', 'it', 'full', 'me
aning', 'complete', 'with', 'the', 'speaker', 'or', 'writer', 's', 'intent', 'a
nd', 'sentiment']
```

It’s possible to improve performance over lemmatization even further if you provide the context in which you want to lemmatize, which you can do through parts-of-speech (POS) tagging.

POS tagging is the task of assigning each word in a sentence the part of speech that it assumes in that sentence. The primary target of POS tagging is to identify the grammatical group of a given word: whether it is a noun, pronoun, adjective, verb, adverbs, etc. based on the context.

References:

1. https://www.geeksforgeeks.org/correcting-words-using-nltk-in-python/
2. https://www.folkstalk.com/tech/how-can-i-corect-word-spelling-by-use-of-nltk-with-code-examples/
3. https://www.analyticsvidhya.com/blog/2021/03/tokenization-and-text-normalization.
4. https://www.geeksforgeeks.org/introduction-to-stemming/

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 4

11.9 PARTS OF SPEECH TAGGING

POS tagging is the task of assigning each word in a sentence the part of speech that it assumes in that sentence. The primary target of POS tagging is to identify the grammatical group of given word: whether it is a noun, pronoun, adjective, verb, adverbs, etc. based on the context.



Most used tags with examples.

- **Noun -** Daniel, London, table, dog, teacher, pen, city, happiness, hope
- **Verb -** go, speak, run, eat, play, live, walk, have, like, are, is
- **Adjective -** big, happy, green, young, fun, crazy, three
- **Adverb -** slowly, quietly, very, always, never, too, well, tomorrow
- **Preposition -** at, on, in, from, with, near, between, about, under
- **Conjunction -** and, or, but, because, so, yet, unless, since, if
- **Pronoun -** I, you, we, they, he, she, it, me, us, them, him, her, this
- **Interjection -** Ouch! Wow! Great! Help! Oh! Hey! Hi!

POS tagging is a supervised learning solution that uses features like the previous word, next word, is first letter capitalized etc. NLTK has a function to get pos tags and it works after tokenization process.

Implementation using NLTK:

```
import nltk
from nltk import word_tokenize
```

```
text = 'I am very hungry but the fridge is empty'
words = word_tokenize(text)
print("Parts of Speech: ",nltk.pos_tag(words))
```

```
Parts of Speech:  [('I', 'PRP'), ('am', 'VBP'), ('very', 'RB'), ('hungry', 'J
J'), ('but', 'CC'), ('the', 'DT'), ('fridge', 'NN'), ('is', 'VBZ'), ('empty',
'JJ')]
```

The following table provides the description of various tags produced using pos_tag() method of NLTK.

Tag	Description	Tag	Description
CC	Coordinating conjunction	PRP$	Possessive pronoun
CD	Cardinal number	RB	Adverb
DT	Determiner	RBR	Adverb, comparative
EX	Existential *there*	RBS	Adverb, superlative
FW	Foreign word	RP	Particle
IN	Preposition or subordinating conjunction	SYM	Symbol
JJ	Adjective	TO	*to*
JJR	Adjective, comparative	UH	Interjection
JJS	Adjective, superlative	VB	Verb, base form
LS	List item marker	VBD	Verb, past tense
MD	Modal	VBG	Verb, gerund or present participle
NN	Noun, singular or mass	VBN	Verb, past participle
NNS	Noun, plural	VBP	Verb, non-3rd person singular present
NNP	Proper noun, singular	VBZ	Verb, 3rd person singular present
NNPS	Proper noun, plural	WDT	Wh-determiner
PDT	Predeterminer	WP	Wh-pronoun
POS	Possessive ending	WP$	Possessive wh-pronoun
PRP	Personal pronoun	WRB	Wh-adverb

11.10 NAMED ENTITY RECOGNITION

Named entity recognition (NER) is one of the most popular data preprocessing tasks. It involves the identification of key information in the text and classification into a set of predefined categories.

NER is used in many fields in Natural Language Processing (NLP), and it can help answering many real-world questions, such as:

- Which companies were mentioned in the news article?
- Were specified products mentioned in complaints or reviews?

- Does the tweet contain the name of a person? Does the tweet contain this person's location?

Some of the categories that are the most important architecture in NER such that:

- Person
- Organization
- Place/ location

Other common tasks include classifying of the following:

- date/time.
- expression
- Numeral measurement (money, percent, weight, etc)
- E-mail address

Implementation using NLTK

Step 1: Import the nltk package, and download all the necessary modules.

```
# Step One: Import nltk and download necessary packages
import nltk
nltk.download('words')
```

```
#Load Data
text="""The Board of Control for Cricket in India (BCCI) is the governing body
for cricket in India and is under the jurisdiction of Ministry of Youth Affairs
and Sports, Government of India. The board was formed in December 1928 as a
society, registered under the Tamil Nadu Societies Registration Act. It is a
consortium of state cricket associations and the state associations select
their representatives who in turn elect the BCCI Chief. Its headquarters are
in Wankhede Stadium, Mumbai. Grant Govan was its first president and
Anthony De Mello its first secretary. """
```

Step 2: In this step,

- First, we return a sentence-tokenised copy of the text using nltk.sent_tokenize(sentence) and iterate over it.
- Next, we tokenise the sentence and find parts of speech of each word; we'll run nltk.pos_tag(nltk.word_tokenize(sent)) individually to see the outputs:
- **Tokenising Words using nltk:** Tokenises the sentences into the list of words
- **POS tagging using nltk:** Identifies parts of speech of each word and returns an array of tuples with the words and their parts of speech.
- **Chunking on POS:** Lastly, we perform a chunking operation that returns a nested nltk.tree. Tree object so that we can iterate or traverse the Tree object to get to the named entities.

- Finally, we get to the output if there is any entity label in the chunk:

```
#Tokenise, find parts of speech and chunk words

for sent in nltk.sent_tokenize(text):
    print(nltk.pos_tag(nltk.word_tokenize(sent)),"\n")
    for chunk in nltk.ne_chunk(nltk.pos_tag(nltk.word_tokenize(sent))):
         if hasattr(chunk, 'label'):
            entity_name = ' '.join(c[0] for c in chunk)
            print(entity_name,":",chunk.label())
    print("\n")
```

```
[('The', 'DT'), ('Board', 'NNP'), ('of', 'IN'), ('Control', 'NNP'), ('for', 'I
N'), ('Cricket', 'NNP'), ('in', 'IN'), ('India', 'NNP'), ('(', '('), ('BCCI',
'NNP'), (')', ')'), ('is', 'VBZ'), ('the', 'DT'), ('governing', 'VBG'), ('bod
y', 'NN'), ('for', 'IN'), ('cricket', 'NN'), ('in', 'IN'), ('India', 'NNP'),
('and', 'CC'), ('is', 'VBZ'), ('under', 'IN'), ('the', 'DT'), ('jurisdiction',
'NN'), ('of', 'IN'), ('Ministry', 'NNP'), ('of', 'IN'), ('Youth', 'NNP'), ('Aff
airs', 'NNPS'), ('and', 'CC'), ('Sports', 'NNP'), (',', ','), ('Government', 'N
NP'), ('of', 'IN'), ('India', 'NNP'), ('.', '.')]

Board : ORGANIZATION
Control : ORGANIZATION
India : GPE
BCCI : ORGANIZATION
India : GPE
Ministry : ORGANIZATION
Youth Affairs : ORGANIZATION
Sports : GPE
India : GPE


[('The', 'DT'), ('board', 'NN'), ('was', 'VBD'), ('formed', 'VBN'), ('in', 'I
N'), ('December', 'NNP'), ('1928', 'CD'), ('as', 'IN'), ('a', 'DT'), ('societ
y', 'NN'), (',', ','), ('registered', 'VBN'), ('under', 'IN'), ('the', 'DT'),
('Tamil', 'NNP'), ('Nadu', 'NNP'), ('Societies', 'NNP'), ('Registration', 'NN
P'), ('Act', 'NNP'), ('.', '.')]

Tamil Nadu : ORGANIZATION


[('It', 'PRP'), ('is', 'VBZ'), ('a', 'DT'), ('consortium', 'NN'), ('of', 'IN'),
('state', 'NN'), ('cricket', 'NN'), ('associations', 'NNS'), ('and', 'CC'), ('t
he', 'DT'), ('state', 'NN'), ('associations', 'NNS'), ('select', 'VBP'), ('thei
r', 'PRP$'), ('representatives', 'NNS'), ('who', 'WP'), ('in', 'IN'), ('turn',
'NN'), ('elect', 'VBP'), ('the', 'DT'), ('BCCI', 'NNP'), ('Chief', 'NNP'),
('.', '.')]

BCCI : ORGANIZATION
```

```
[('Its', 'PRP$'), ('headquarters', 'NNS'), ('are', 'VBP'), ('in', 'IN'), ('Wank
hede', 'NNP'), ('Stadium', 'NNP'), (',', ','), ('Mumbai', 'NNP'), ('.', '.')]

Wankhede Stadium : GPE
Mumbai : GPE


[('Grant', 'NNP'), ('Govan', 'NNP'), ('was', 'VBD'), ('its', 'PRP$'), ('first',
'JJ'), ('president', 'NN'), ('and', 'CC'), ('Anthony', 'NNP'), ('De', 'NNP'),
('Mello', 'NNP'), ('its', 'PRP$'), ('first', 'JJ'), ('secretary', 'NN'), ('.',
'.')]

Grant : PERSON
Govan : PERSON
Anthony De Mello : PERSON
```

References:

1. https://medium.com/mlearning-ai/nlp-tokenization-stemming-lemmatization-and-part-of-speech-tagging-9088ac068768
2. https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html
3. https://nanonets.com/blog/named-entity-recognition-with-nltk-and-spacy
4. https://www.geeksforgeeks.org/named-entity-recognition
5. https://analyticsindiamag.com/guide-to-named-entity-recognition-with-spacy-and-nltk/

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 5

11.11 VECTORIZER

Vectorization or word embedding is jargon for a classic approach of converting input data from its raw format (i.e., text) into vectors of real numbers which is the format that Machine Learning models support.

- After text cleaning and normalization, the processed text is converted to feature vectors so that we can feed it to machine learning applications.
- In Machine Learning, vectorization is a step, in feature extraction. The idea is to get some distinct features out of the text for the model to train on, by converting text to numerical vectors.

Why do we need Word Embeddings?

- Many Machine Learning algorithms and almost all Deep Learning Architectures are not capable of processing strings or plain text in their raw form.
- They require numerical numbers as inputs to perform any sort of task, such as classification, regression, clustering, etc.
- Also, from the huge amount of data that is present in the text format, it is imperative to extract some knowledge out of it and build any useful applications.
- In short, to build any model in machine learning or deep learning, the final level data has to be in numerical form because models don't understand text or image data directly as humans do.

Terminologies

- **Document:** A document is a single text data point. For Example, a review of a particular product by the user.
- **Corpus:** It a collection of all the documents present in our dataset.
- **Feature:** Every unique word in the corpus is considered as a feature.

For Example, Let's consider the 2 documents shown below:

D1: Dog hates a cat. It loves to go out and play.

D2: Cat loves to play with a ball.

We can build a corpus from the above 2 documents just by combining them.

Corpus = "Dog hates a cat. It loves to go out and play. Cat loves to play with a ball."

And features will be all unique words:

Features: ['and', 'ball', 'cat', 'dog', 'go', 'hates', 'it', 'loves', 'out', 'play', 'to', 'with']

11.11.1 Types of Vectorization/Word Embeddings

1. One-Hot Encoding (OHE):

In this technique, we represent each unique word in vocabulary by setting a unique token with value 1 and rest 0 at other positions in the vector. In simple words, a vector representation of a one-hot encoded vector represents in the form of 1, and 0 where 1 stands for the position where the word exists and 0 everywhere else.

Let's consider the following sentence:

Sentence: I am teaching NLP in Python

A word in this sentence may be "NLP", "Python", "teaching", etc.

Since a dictionary is defined as the list of all unique words present in the sentence. So, a dictionary may look like –

Dictionary: **['I', 'am', 'teaching',' NLP',' in', 'Python']**

Therefore, the vector representation in this format according to the above dictionary is

Vector for NLP: [0,0,0,1,0,0]

Vector for Python: [0,0,0,0,0,1]

This is just a very simple method to represent a word in vector form.

Disadvantages of One-hot Encoding

1. One of the disadvantages of One-hot encoding is that the Size of the vector is equal to the count of unique words in the vocabulary.

2. One-hot encoding does not capture the relationships between different words. Therefore, it does not convey information about the context.

2. Count Vectorizer

- It is one of the simplest ways of doing text vectorization.
- It creates a document term matrix, which is a set of dummy variables that indicates if a particular word appears in the document.
- Count vectorizer will fit and learn the word vocabulary and try to create a document term matrix in which the individual cells denote the frequency of that word in a particular document, which is also known as term frequency, and the columns are dedicated to each word in the corpus.

Matrix Formulation

Consider a Corpus C containing D documents $\{d_1,d_2…..d_D\}$ from which we extract N unique tokens. Now, the dictionary consists of these N tokens, and the size of the Count Vector matrix M formed is given by D X N. Each row in the matrix M describes the frequency of tokens present in the document D_i.

Let's consider the following example:

Document-1: He is a smart boy. She is also smart.

Document-2: Chirag is a smart person.

The dictionary created contains the list of unique tokens(words) present in the corpus

Unique Words: ['He', 'She', 'smart', 'boy', 'Chirag', 'person']

Here, D=2, N=6

So, the count matrix M of size 2 X 6 will be represented as –

	He	She	smart	boy	Chirag	person
D1	1	1	2	1	0	0
D2	0	0	1	0	1	1

Now, a column can also be understood as a word vector for the corresponding word in the matrix M.

For Example, for the above matrix formed, let's see the word vectors generated.

Vector for '**smart'** is **[2,1],**

Vector for '**Chirag'** is **[0, 1],** and so on.

Implementation:

```
from sklearn.feature_extraction.text import CountVectorizer
sents = ['He is a smart boy. She is also smart',
         'Chirag is a smart person']
```

```
cv = CountVectorizer()
X = cv.fit_transform(sents)
X = X.toarray()
vocabulary = sorted(cv.vocabulary_.keys())
print(vocabulary)
print(X)
```

```
['also', 'boy', 'chirag', 'he', 'is', 'person', 'she', 'smart']
[[1 1 0 1 2 0 1 2]
 [0 0 1 0 1 1 0 1]]
```

3. N-grams

N-grams are contiguous sequences of n-items in a sentence. N can be 1, 2 or any other positive integers, although usually we do not consider very large N because those n-grams rarely appears in many different places. When performing machine learning tasks related to natural language processing, we usually need to generate n-grams from input sentences.

For example, in text classification tasks, in addition to using each individual token found in the corpus, we may want to add bi-grams or tri-grams as features to represent our documents.

Implementation:

NLTK has the ngrams() function that returns a generator of n-grams given a tokenized sentence.

```
import re
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

text = """Natural language processing refers to the branch of
          computer science concerned with giving computers
          the ability to understand text"""
words = word_tokenize(text)
output = list(ngrams(words, 2))
output
```

```
[('Natural', 'language'),
 ('language', 'processing'),
 ('processing', 'refers'),
 ('refers', 'to'),
 ('to', 'the'),
 ('the', 'branch'),
 ('branch', 'of'),
 ('of', 'computer'),
 ('computer', 'science'),
 ('science', 'concerned'),
 ('concerned', 'with'),
 ('with', 'giving'),
 ('giving', 'computers'),
 ('computers', 'the'),
 ('the', 'ability'),
 ('ability', 'to'),
 ('to', 'understand'),
 ('understand', 'text')]
```

Similar to the count vectorization technique, in the N-Gram method, a document term matrix is generated and each cell represents the count. The difference in the N-grams method is that the count represents the combination of adjacent words of length n in the title. Count vectorization is N-Gram where n=1. For example, “I love this article” has four words and n=4.

- if n=2, i.e bigram, then the columns would be — [“I love”, “love this”, ‘this article”]
- if n=3, i.e trigram, then the columns would be — [“I love this”, ”love this article”]
- if n=4,i.e four-gram, then the column would be -[‘“I love this article”]

The n value is chosen based on performance.

```
cv = CountVectorizer(ngram_range=(2,2))
X = cv.fit_transform(sents)
X = X.toarray()
vocabulary = sorted(cv.vocabulary_.keys())
print(vocabulary)
print(X)
```

```
['also smart', 'boy she', 'chirag is', 'he is', 'is also', 'is smart', 'she i
s', 'smart boy', 'smart person']
[[1 1 0 1 1 1 1 1 0]
 [0 0 1 0 0 1 0 0 1]]
```

References:

1. https://www.analyticsvidhya.com/blog/2021/06/part-5-step-by-step-guide-to-master-nlp-text-vectorization-approaches/
2. https://neptune.ai/blog/vectorization-techniques-in-nlp-guide
3. https://towardsdatascience.com/nlp-in-python-vectorizing-a2b4fc1a339e
4. https://albertauyeung.github.io/2018/06/03/generating-ngrams.html/
5. Infosys Springboard – Natural Language Processing for developers

WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 6

11.12 IF-TDF VECTORIZATION

All the methods discussed in the previous sessions were based on the Bag of Words model which is simple and works well. But the problem with BoW is that it treats all words equally. As a result, it cannot distinguish very common words or rare words. So, to solve this problem, TF-IDF comes into the picture. TF-IDF is made up of two terms: Term Frequency and Inverse Document Frequency

1. Term Frequency

Term frequency denotes the frequency of a word in a document. For a specified word, it is defined as the ratio of the number of times a word appears in a document to the total number of words in the document. Or, it is also defined in the following manner:

It is the percentage of the number of times a word (x) occurs in a particular document (y) divided by the total number of words in that document.

$$TF(\boldsymbol{term}) = \frac{Number\ of\ times\ \boldsymbol{term}\ appears\ in\ a\ document}{Total\ number\ of\ items\ in\ the\ document}$$

For Example, Consider the following sentence

'Cat loves to play with ball'

For the above sentence, the term frequency value for word cat will be: tf('cat') = 1/6

This number will always stay ≤ 1, thus we now judge how frequent a word is in the context of all of the words in a document.

2. Inverse document frequency

Inverse document frequency looks at how common (or uncommon) a word is amongst the corpus. It measures the importance of the word in the corpus.

Before we go into IDF, we must make sense of DF – Document Frequency. It's given by the following formula:

$$DF = \frac{Documents\ containing\ word\ W}{Total\ number\ of\ documents}$$

DF tells us about the proportion of documents that contain a certain word. IDF is the reciprocal of the Document Frequency.

$$IDF = log\left(\frac{Total\ number\ of\ documents}{Documents\ containing\ word\ W}\right)$$

The intuition behind using IDF is that the more common a word is across all documents, the lesser its importance is for the current document. A logarithm is taken to dampen the effect of (normalize) IDF in the final calculation. The final TF-IDF score comes out to be:

$$TFIDF = TF * IDF$$

Implementation

```
from sklearn.feature_extraction.text import TfidfVectorizer
sents = ['coronavirus is a highly infectious disease',
   'coronavirus affects older people the most',
   'older people are at high risk due to this disease']
```

```
tfidf = TfidfVectorizer()
transformed = tfidf.fit_transform(sents)

import pandas as pd
df = pd.DataFrame(transformed[0].T.todense(),
                  index=tfidf.get_feature_names_out(),
                  columns=["TF-IDF"])
df = df.sort_values('TF-IDF', ascending=False)
df
```

	TF-IDF
infectious	0.490479
highly	0.490479
is	0.490479
coronavirus	0.373022
disease	0.373022
older	0.000000
this	0.000000
the	0.000000
risk	0.000000
people	0.000000
affects	0.000000
most	0.000000
are	0.000000
high	0.000000
due	0.000000
at	0.000000
to	0.000000

References:

1. https://www.analyticsvidhya.com/blog/2021/06/part-5-step-by-step-guide-to-master-nlp-text-vectorization-approaches/
2. https://neptune.ai/blog/vectorization-techniques-in-nlp-guide
3. https://towardsdatascience.com/nlp-in-python-vectorizing-a2b4fc1a339e
4. Infosys Springboard – Natural Language Processing for developers

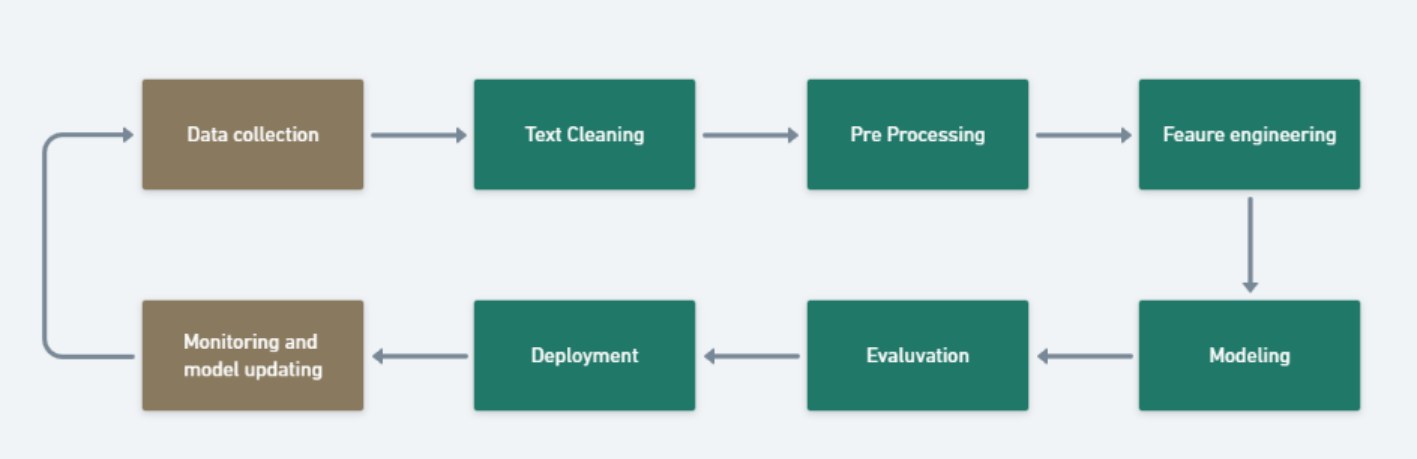
WEEK- 11

NATURAL LANGUAGE PROCESSING

Session 7

11.13 NLP Pipeline

The set of ordered stages one should go through from a labeled dataset to creating a classifier that can be applied to new samples is called the NLP pipeline. NLP Pipeline is a set of steps followed to build an end to end NLP software.



11.13.1 Steps involved in building NLP Pipeline

1. Data Acquisition

In the data acquisition step, we collect the data required for building our NLP software. We can collect the data using any of the following methods:

a) We can conduct to survey to collect data and then manually give a label to the data
b) Public Dataset – If a public dataset is available for our problem statement.
c) Web Scrapping – Scrapping data using beautiful soup or other libraries

2. Text Preprocessing

Once the data collection step is done, we cannot use this data as is for model building. We have to do text preprocessing. It helps to remove unhelpful parts of the data, or noise, by converting all characters to lowercase, removing stop words, punctuation marks, and typos in the data. After doing data preprocessing accuracy of the model get increases.

Steps involved in Text Preprocessing –

1. Text Cleaning – In-text cleaning we do HTML tag removing, removing punctuations, Spelling checker, etc.

2. Basic Preprocessing — In basic preprocessing we do tokenization (word or sent tokenization), stop word removal, removing digits, lower casing etc.

3. Advance Preprocessing — In this step we do POS tagging, Named entity recognition etc.

3. Feature Engineering

After text cleaning and normalization, the processed text is converted to feature vectors so that we can feed it to machine learning applications. Feature Engineering means converting text data to numerical data. But why it is required to convert text data to numerical data? Because many Machine Learning algorithms and almost all Deep Learning Architectures are not capable of processing strings or plain text in their raw form. This step is also called Feature extraction from text.

In this step, we use multiple techniques to convert text to numerical vectors.

1. One Hot Encoder
2. Bag Of Word(BOW)
3. n-grams
4. Tf-Idf
5. Word2vec

4. Modelling / Model Building

In the modeling step, we try to create a model based on the cleaned data. Here also we can use multiple approaches to build the model based on the problem statement.

Approaches to building model –

1. Heuristic Approach
2. Machine Learning Approach
3. Deep Learning Approach

5. Model Evaluation

In the model evaluation, we can use different metrics for evaluation such as Accuracy, Recall, Confusion Metrics, Perplexity, etc.

6. Deployment

In the deployment step, we have to deploy our model on the cloud for the users. Deployment has three stages deployment, monitoring, and retraining or model update.

Three stages of deployment –

1. **Deployment** – model deploying on the cloud for users.
2. **Monitoring** – In the monitoring phase, we have to watch the model continuously. Here we have to create

a dashboard to show evaluation metrics.

3. **Update**- Retrain the model on new data and again deploy.

References:

1. https://www.analyticsvidhya.com/blog/2022/05/nlp-preprocessing-steps-in-easy-way/
2. https://libguides.library.usyd.edu.au/text_data_mining/cleaning
3. https://www.analyticsvidhya.com/blog/2022/01/text-cleaning-methods-in-nlp/
4. https://www.analyticsvidhya.com/blog/2022/06/an-end-to-end-guide-on-nlp-pipeline
5. Infosys Springboard – Natural Language Processing for developers